МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ

по лабораторной работе №2
по дисциплине «Построение и анализ алгоритмов»
ТЕМА: Кратчайшие пути в графе: коммивояжёр.
Вариант: 7

Студент гр. 4343 _____ Лапшин А.Е.

Преподаватель _____ Жангиров Т. Р.

Г. Санкт-Петербург
2026 г.

Задание

Напишите программу, решающую задачу коммивояжёра. Нужно найти кратчайший маршрут, который проходит через все заданные города ровно один раз и возвращается в исходный город. Не все города могут быть напрямую связаны друг с другом.

Входные данные:

- n – количество городов ($5 \leq n \leq 15$).
- Матрица расстояний между городами размером $n \times n$, где $graph[i][j]$ обозначает расстояние от города i до города j. Если $graph[i][j] = 0$ и $(i \neq j)$, это означает, что прямого пути между городами нет.

Выходные данные:

- Минимальная стоимость маршрута, проходящего через все города и возвращающегося в начальный город.
- Оптимальный путь в виде последовательности посещаемых городов, начинающейся и заканчивающейся в городе 0.
- Если такого пути не существует, вывести "no path".

Sample Input 1:

5

0 1 13 23 7

12 0 15 18 28

21 29 0 33 28

23 19 34 0 38

5 40 7 39 0

Sample Output 1:

78

0 4 2 3 1 0

Sample Input 2:

3

0 1 0

1 0 1

0 1 0

Sample Output 2:

no path

Вариант 7

Точный метод: динамическое программирование (не МВиГ), рекурсивная реализация.

Приближённый алгоритм: АЛШ-1.

Требование перед сдачей: прохождение кода в задании 3.1 на Stepik. Замечание к варианту 7. АЛШ-1 начинать со стартовой вершины.

Выполнение работы

Программа реализует два подхода к решению задачи коммивояжёра:

1) Точный алгоритм Хелда Карпа на основе динамического программирования.

2) Приближённый эвристический, алгоритм АЛШ-1.

Оба алгоритма принимают на вход количество вершин и матрицу смежности.

Точный алгоритм (алгоритм Хелда Карпа)

Основные структуры данных:

vector<vector<int>> matrix_orig – двумерный вектор размером $n \times n$. Является матрицей смежности графа. Каждый элемент matrix_orig $[i][j]$ содержит вес ребра из вершины i в вершину j. Если ребро отсутствует, значение равно INFINITY. Данная структура является входными данными.

vector<vector<int>> matrix_mask_last – двумерный вектор размером $(1 << n) \times n$, изначально заполненный -1. Элемент matrix_mask_last$[mask][last]$ содержит минимальную стоимость маршрута из нулевой вершины в вершину last и содержащий еще вершины, отмеченные 1 в битовой маске.

vector<vector<int>> matrix_path – двумерный вектор размером $(1 << n) \times n$, изначально заполненный -1. Используется для восстановления итогового пути. Элемент matrix_path$[mask][pos]$ хранит номер следующей вершины, которую нужно посетить после вершины pos для конкретного случая битовой маски, чтобы достичь минимальной стоимости маршрута.

Алгоритм Хелда Карпа:

Функция recursion(mask, last):

Находит минимальную стоимость завершения маршрута, если уже посещены все вершины из mask, алгоритм в вершине last и требуется вернуться в вершину 0, посетив все оставшиеся вершины.

Принцип работы:

1) Проверка, что все вершины посещены.

Если mask равен (1 << n) - 1 (то есть все n вершин уже посещены), функция возвращает стоимость ребра из текущей вершины last обратно в стартовую вершину 0 (matrix_orig[last][0]).

2) Проверка мемоизации.

Если значение для пары (mask, last) уже было вычислено ранее (хранится в matrix_mask_last), то мы выходим из данного случая. То есть каждое состояние вычисляется только один раз.

3) Рекурсивный перебор вариантов.

Функция перебирает все вершины point от 0 до n-1. Для каждой вершины проверяются два условия:

- point_not_in_mask – вершина ещё не посещена (соответствующий бит в маске равен 0).
- path_exists – существует прямое ребро из текущей вершины last в point (значение в матрице не равно INFINITY).

Если оба условия выполнены, функция:

1. Создаёт новую маску new_mask = mask | (1 << point), добавляя вершину point в множество посещённых.
2. Рекурсивно вызывает recursion(new_mask, point), получая минимальную стоимость завершения маршрута из нового состояния.
3. Вычисляет полную стоимость варианта как matrix_orig[last][point] + recursion(new_mask, point).
4. Сравнивает полученную стоимость с текущим минимумом и, если она меньше, обновляет min_cost и запоминает point как best_next_point.

Функция reconstr_path():

Восстанавливает оптимальный маршрут по сохранённой таблице matrix_path. Она работает следующим образом:

1) Инициализирует path списком, содержащим стартовую вершину 0.
2) Устанавливает начальную маску mask = 1 << 0 (посещена только вершина 0) и текущую позицию last = 0.
3) В цикле while (mask != full_mask) (пока не посещены все вершины):
    1. Извлекает следующую вершину next = matrix_path[mask][last].
    2. Добавляет next в путь.
    3. Обновляет маску: mask |= (1 << next).
    4. Обновляет текущую позицию: last = next.
4) После завершения цикла добавляет возврат в стартовую вершину 0.
5) Возвращает восстановленный маршрут.

Функция main()

Считывает из стандартного потока ввода целое число n - количество вершин графа, а затем матрицу смежности размером n × n. Все нулевые значения (кроме диагональных) заменяются на INFINITY, что обозначает отсутствие ребра. Инициализирует matrix_mask_last и matrix_path, запускает рекурсивный поиск и после функцию восстановления пути, который выводит на экран с минимальной стоимостью маршрута.

Оценка сложности

Временная сложность: $O(n^2 \cdot 2^n)$ – каждое из $n \cdot 2^n$ состояний обрабатывается один раз, и для каждого перебирается до n возможных следующих вершин. 2^n – количество возможных масок.

Пространственная сложность: $O(n \cdot 2^n)$ для хранения таблиц matrix_mask_last и matrix_path.

Приближенный алгоритм (АЛШ-1)

Основные структуры данных:

matrix - матрица смежности графа размером n × n. Каждый элемент matrix[i][j] содержит вес ребра из вершины i в вершину j. Если ребро отсутствует, значение равно INFINITY. Данная структура является входными данными.

visited - булев вектор размера n, где visited[i] == true означает, что вершина i уже посещена и включена в маршрут.

path - вектор целых чисел, хранящий текущий построенный маршрут (последовательность вершин).

Алгоритм АЛШ-1:

Функция lower_bound(matrix, visited, path, n)

Вычисляет нижнюю границу стоимости оставшегося пути - минимально возможную стоимость, за которую можно завершить маршрут, если из текущего состояния продолжить оптимальным образом.

Принцип работы:

1. Определяет начальную вершину маршрута start = path[0] (всегда 0) и конечную вершину end = path.back() (текущая позиция).
2. Ищет **минимальное входящее ребро** в стартовую вершину start из всех **непосещённых** вершин – сохраняет в min_in.
3. Ищет **минимальное исходящее ребро** из конечной вершины end во все **непосещённые** вершины – сохраняет в min_out.
4. Вычисляет и возвращает нижнюю границу по формуле: lower = (min_in + min_out) / 2.0

Функция main()

1) Чтение входных данных

Считывает из стандартного потока ввода целое число n – количество вершин графа, а затем матрицу смежности размером n × n. Все

нулевые значения (кроме диагональных) заменяются на INFINITY, что обозначает отсутствие ребра.

2) Инициализация структур

- Создаётся булев вектор visited размера n, изначально все вершины отмечены как непосещённые.
- Создаётся вектор path, в который добавляется стартовая вершина **0**.
- Вершина 0 отмечается как посещённая (visited[0] = true).
- Устанавливаются начальные значения: last = 0 (текущая позиция), total_cost = 0 (накопленная стоимость).

3) Основной цикл построения маршрута

Цикл выполняется, пока path.size() < n (пока не посещены все вершины).

Если осталась только одна непосещённая вершина:

- Находится единственная непосещённая вершина next.
- Проверяется существование ребра из last в next.
- Добавляется эта вершина в путь, обновляется стоимость и завершается основной цикл.

Иначе:

Для каждой вершины next, которая ещё не посещена и достижима из last:

1. Создаёт временные копии visited и path, добавляя next в путь.
2. Вычисляет нижнюю границу L = lower_bound(temp_visited, temp_path, n) для состояния после перехода в next.
3. Вычисляет оценочную стоимость варианта: value = matrix[last][next] + L
4. Сравнивает value с текущим лучшим значением и запоминает next с минимальной оценкой.

После перебора всех вариантов выбирается вершина best_next с наименьшим значением value.

Происходит переход в выбранную вершину

- Вершина best_next добавляется в path.
- total_cost увеличивается на стоимость перехода.
- best_next отмечается как посещённая.
- Текущая позиция last обновляется.

После посещения всех вершин:

- Проверяется наличие ребра из последней вершины last обратно в вершину 0.
- Добавляет стоимость возврата к total_cost.
- Добавляет вершину 0 в конец path.

4) Вывод результата

Оценка сложности

Временная сложность: O(n³) - на каждом из n-1 шагов для каждой из O(n) вершин-кандидатов вычисляется lower_bound, которая выполняет O(n) операций для поиска минимальных рёбер.

Пространственная сложность: $\boldsymbol{O(n^2)}$ - для хранения векторов visited и path требуется $\boldsymbol{O(n)}$. А для матрицы размером n**:** $\boldsymbol{O(n^2)}$

Тестирование

Точный алгоритм – алгоритм Хелда Карпа

Входные данные	Выходные данные
5 0 1 13 23 7 12 0 15 18 28 21 29 0 33 28 23 19 34 0 38 5 40 7 39 0	5 0 2 1 0
7 76 70 5 18 38 18 67 12 80 40 43 43 79 17 8 52 43 60 27 41 18 48 70 30 10 63 33 55 82 26 35 84 73 33 97 68 6 52 89 22 17 84 26 84 70 24 45 86 7	140 0 2 6 3 5 4 1 0
5 0 0 0 0 0 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1	no path
1 1	1 0 0
15 17 84 41 87 58 99 20 63 28 67 97 55 69 27 54 7 16 16 84 14 53 73 26 74 64 44 93 4 78 98 9 14 92 85 78 74 84 3 75 66 72 32 95 87 2 80 20 35 4 73 44 26 16 67 8 62 58 58 18 50 16 60 84 19 58 28 45 86 35 0 41 57 35 57 59 34 69 91 6 76 42 49 10 87 18 22 20 3 72 67 89 20 28 66 32 16 60 49 72 63 44 28 49 80 23	200 0 6 1 4 3 9 10 7 5 11 12 13 2 14 8 0

10 12 44 38 64 8 56 44 90 53 68 51 2 28 8 13 86 74 41 92 61 85 99 54 65 22 94 78 38 69 23 51 7 59 66 8 25 54 15 28 9 22 37 85 56 16 51 85 55 57 100 71 8 70 53 23 85 46 97 24 51 83 66 42 93 37 76 96 74 73 69 53 13 40 36 52 33 55 7 78 75 31 91 22 52 54 63 54 24 881 65 99 16 88 21 31 87 29 42 67 97 77 99 10 73 30 60 44 62 64 0 70 66 6 66 49 89 76 56 85	

Приближенный алгоритм (АЛШ-1)

Входные данные	Выходные данные
15 62 97 94 76 79 92 11 45 44 52 79 43 31 7 73 70 55 96 3 94 19 90 100 47 47 15 7 45 1 84 89 17 56 81 38 99 89 76 42 26 96 95 89 34 78 49 47 1 70 2 93 88 92 11 90 76 64 14 55 55 50 15 70 65 12 3 84 40 35 71 88 80 62 25 68 54 36 13 60 3 27 10 46 36 64 10 28 8 35 26 7 69 70 50 1 83 64 32 44 99 62 67 41 34 12 54 5 86 55 83 48 97 21 39 78 55 51 32 78 83 94 61 11 89 36 33 47 61 50 50 70 98 8 94 86 32 97 78 4 81 36 23 95 60 94 6 42 61 50 38 8 42 35 80 63 60 26 28 99 72 93 52 19 87 82 66 14 54 63 13 18 48 35 50 68 97 90 5 91 10	321 0 13 1 3 4 5 6 14 2 9 10 12 8 7 11 0

62 89 61 69 78 88 53 68 49 98 97 82 37 57 57 73 2 90 91 21 46 43 76 72 99 11 33 82 64 65 14 70 4 32 35 65 97 56 19 61 4 55 60 70 52	
3 0 0 1 2 3 0 5 2 0	5 0 2 1 0
5 0 0 0 0 0 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1	no path
1 1	1 0 0
15 17 84 41 87 58 99 20 63 28 67 97 55 69 27 54 7 16 16 84 14 53 73 26 74 64 44 93 4 78 98 9 14 92 85 78 74 84 3 75 66 72 32 95 87 2 80 20 35 4 73 44 26 16 67 8 62 58 58 18 50 16 60 84 19 58 28 45 86 35 0 41 57 35 57 59 34 69 91 6 76 42 49 10 87 18 22 20 3 72 67 89 20 28 66 32 16 60 49 72 63 44 28 49 80 23 10 12 44 38 64 8 56 44 90 53 68 51 2 28 8 13 86 74 41 92 61 85 99 54 65 22 94 78 38 69 23 51 7 59 66 8 25 54 15 28 9 22 37 85 56 16 51 85 55 57 100 71 8 70 53 23 85 46 97 24	287 0 6 5 12 3 9 2 14 8 10 7 1 4 11 13 0

51 83 66 42 93 37 76 96 74 73 69 53 13 40 36 52 33 55 7 78 75 31 91 22 52 54 63 54 24 881 65 99 16 88 21 31 87 29 42 67 97 77 99 10 73 30 60 44 62 64 0 70 66 6 66 49 89 76 56 85	

Вывод

В ходе выполнения лабораторной работы были разработаны и реализованы два алгоритма решения задачи коммивояжёра: точный алгоритм Хелда-Карпа, основанный на динамическом программировании по битовым маскам, и приближённый алгоритм АЛШ-1, использующий нижнюю границу для оценки качества выбора следующей вершины.

Приложение

Код программы:

ALSH-1.cpp

```
#include <iostream>
#include <vector>

using namespace std;

int INFINITY = 1e5;


double lower_bound(vector<vector<int>>& matrix,
vector<bool>& visited, vector<int>& path, int n) {
    int start = path[0];
    int end = path.back();

#ifdef LOG
    cout << "Ищем нижнюю оценку дороги из вершины " << start
<< " в " << end << endl;
#endif

    int min_in = INFINITY;
    int min_out = INFINITY;

    for (int i = 0; i < n; i++) {
        if (!visited[i]) {

#ifdef LOG
            cout << "Вершина " << i << ": ";
             cout << "   ребро в " << start << " = " <<
matrix[i][start];
             cout << "   ребро из " << end << " = " <<
matrix[end][i] << endl;
#endif
```

```
            if (matrix[i][start] < min_in) {
                min_in = matrix[i][start];
            }
            if (matrix[end][i] < min_out) {
                min_out = matrix[end][i];
            }
        }
    }

    int lower = (min_in + min_out) / 2;

#ifdef LOG
    cout << "Нижняя граница " << lower << endl;
#endif

    return lower;
}



int main() {
    int n;
    cin >> n;

    vector<vector<int>> matrix(n, vector<int>(n));
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            cin >> matrix[i][j];
            if (matrix[i][j] == 0) {
                matrix[i][j] = INFINITY;
            }
        }
    }

    vector<bool> visited(n, false);
```

```
        vector<int> path;

        path.push_back(0);
        visited[0] = true;
        int last = 0;
        int total_cost = 0;

    #ifdef LOG
        cout << "Стартовая вершина: 0" << endl;
        cout << "Начальный путь: 0" << endl;
        cout << "Начальная стоимость: 0" << endl;
    #endif

        while (path.size() < n) {

    #ifdef LOG
            cout << "Текущий путь: ";
            for (int v : path) cout << v << " ";
            cout << "\nТекущая позиция: " << last << endl;
            cout << "Накопленная стоимость: " << total_cost <<
endl;
            cout << "Осталось посетить: " << n - path.size() <<
" вершин" << endl;
            cout << endl;
    #endif


            if (path.size() == n - 1) {
    #ifdef LOG
                cout << "Осталась последняя вершина" << endl;
    #endif
                int next = -1;
                for (int i = 0; i < n; i++) {
                    if (!visited[i]) {
                        next = i;
```

```
                        break;
                    }
                }


                if (next == -1 || matrix[last][next] == INFINITY)
{
                    cout << "no path" << endl;
                    return 0;
                }

     #ifdef LOG
                 cout << "Последняя непосещенная вершина: " <<
next << endl;
                 cout << "Стоимость перехода из " << last << "
в " << next << ": " << matrix[last][next] << endl;
     #endif
                path.push_back(next);
                total_cost += matrix[last][next];
                last = next;
                break;
            }


            double best_value = INFINITY;
            int best_next = -1;

            for (int next = 0; next < n; next++) {
                  if (!visited[next] && matrix[last][next] !=
INFINITY) {

     #ifdef LOG
                    cout << "\nРассматриваем вершину " << next
<< endl;
```

```
                    cout << "  Стоимость перехода из " << last
<< " в " << next << ": " << matrix[last][next] << endl;
     #endif

                    vector<bool> temp_visited = visited;
                    vector<int> temp_path = path;

                    temp_visited[next] = true;
                    temp_path.push_back(next);

                   double L = lower_bound(matrix, temp_visited,
temp_path, n);

                    double value = matrix[last][next] + L;

                    if (value < best_value) {
     #ifdef LOG
                      cout << "старая лучшая цена равна " <<
best_value << endl;
                   cout << "новая лучшая цена равна " << value
<< endl;
     #endif
                        best_value = value;
                        best_next = next;
                    }
                }
            }

            if (best_next == -1) {
                cout << "no path" << endl;
                return 0;
            }

            path.push_back(best_next);
            total_cost += matrix[last][best_next];
```

```
            visited[best_next] = true;
            last = best_next;
        }


    #ifdef LOG
        cout << "Текущий путь (все вершины посещены): ";
        for (int v : path) cout << v << " ";
        cout << endl;
        cout << "Последняя вершина: " << last << endl;
        cout << "Стоимость возврата " << last << " в 0: " <<
matrix[last][0] << endl;
    #endif

        if (matrix[last][0] == INFINITY) {
            cout << "no path" << endl;
            return 0;
        }

        total_cost += matrix[last][0];
        path.push_back(0);

        cout << total_cost << endl;
        for (size_t i = 0; i < path.size(); i++) {
            cout << path[i];
            if (i < path.size() - 1) cout << " ";
        }
        cout << endl;

        return 0;
    }
```

main.cpp

```
#include <iostream>
#include <vector>

using namespace std;

int INFINITY = 1e5;
vector<vector<int>> matrix_orig;
vector<vector<int>> matrix_mask_last;
vector<vector<int>> matrix_path;
int n;

int recursion(int mask, int last) {
    int full_mask = (1 << n) - 1;
#ifdef LOG
    cout << "\nrecursion: mask = " << mask << ", last = "
<< last << endl;
#endif

    if (mask == full_mask) {
#ifdef LOG
        cout << "  Все вершины посещены" << endl;
#endif
        return matrix_orig[last][0];
    }

    if (matrix_mask_last[mask][last] != -1) {
#ifdef LOG
        cout << "  Этот случай уже вычислен вычислен!
matrix_mask_last[" << mask << "][" << last << "] = " <<
matrix_mask_last[mask][last] << endl;
#endif
        return matrix_mask_last[mask][last];
    }

    int min_cost = INFINITY;
    int best_next_point = -1;

    for (int point = 0; point < n; point++) {

        bool puth_exists = matrix_orig[last][point] !=
INFINITY;
        bool point_not_in_mask = !(mask & (1 << point));

        if (point_not_in_mask && puth_exists) {

#ifdef LOG
            cout << "   Рассматриваем переход из " << last
<< " в " << point;
            cout << "    Стоимость перехода = " <<
matrix_orig[last][point] << endl;
#endif
```

```
                int new_mask = mask | (1 << point);
                     int  cost  =  matrix_orig[last][point]  +
recursion(new_mask, point);

     #ifdef LOG
                cout << "    Стоимость через " << point << " =
" << cost << endl;
     #endif

                if (cost < min_cost) {

     #ifdef LOG
                    cout << "Найдено новое лучшее решение! Было
" << min_cost << ", стало " << cost << endl;
     #endif

                    min_cost = cost;
                    best_next_point = point;
                }

            }
        }

     #ifdef LOG
         cout << "  Лучший переход из " << last << " в " <<
best_next_point << " стоит " << min_cost << endl;;
        cout << "  Сохраняем matrix_path[" << mask << "][" <<
last << "] = " << best_next_point << endl;
        cout << "  Сохраняем matrix_mask_last[" << mask << "]["
<< last << "] = " << min_cost << endl;
     #endif

        matrix_path[mask][last] = best_next_point;

        matrix_mask_last[mask][last] = min_cost;
        return min_cost;
     }



     vector<int> reconstr_puth() {
     #ifdef LOG
        cout << "\nВосстанавливаем путь" << endl;
     #endif
        vector<int> path;
        path.push_back(0);
        int mask = 1 << 0;
        int last = 0;
        int full_mask = (1 << n) - 1;

        while (mask != full_mask) {
            int next = matrix_path[mask][last];
```

```
#ifdef LOG
        cout << "mask = " << mask << ", last = " << last <<
endl;
         cout << "matrix_path[" << mask << "][" << last <<
"] = " << next << endl;
#endif
        path.push_back(next);
        mask |= (1 << next);
        last = next;
#ifdef LOG
        cout << "Добавляем " << next << " в путь" << endl;
        cout << "Новая маска: " << mask << endl;
        cout << "Текущий путь: ";
        for (int v : path) cout << v << " ";
#endif
    }

    path.push_back(0);

#ifdef LOG
    cout << "\nДобавляем возврат в 0" << endl;
#endif

    return path;
}

int main() {
    cin >> n;

    matrix_orig.assign(n, vector<int>(n));
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            cin >> matrix_orig[i][j];
            if (matrix_orig[i][j] == 0) {
                matrix_orig[i][j] = INFINITY;
            }
        }
    }

    matrix_mask_last.assign(1 << n, vector<int>(n, -1));
    matrix_path.assign(1 << n, vector<int>(n, -1));

    int start_mask = 1 << 0;
    int result = recursion(start_mask, 0);

    if (result == INFINITY) {
        cout << "no path";
        return 0;
    }

    vector<int> full_path = reconstr_puth();

    cout << result << endl;
```

```
    for (int i = 0; i < full_path.size(); i++) {
        cout << full_path[i];
        if (i < full_path.size() - 1) cout << " ";
    }
    cout << endl;

    return 0;
}
```